

Teacher:	Ray Joshua Balingkit	**Subject:**	Computer Science
Week:	1 (Term 1)	**Year:**	10-11
Link:		**Cover:**	

LESSON OVERVIEW

What should students know/do by the end of this week's lessons?

Students will begin the course by establishing safe and respectful computer-room routines and developing their understanding of problems and algorithms. They will break a familiar computing task into smaller parts and explain what each part needs to achieve. Students will then write precise, ordered instructions and improve them through partner testing, recognising why a computer requires more explicit instructions than a person may need.

End-of-Week Assessment/Check: How will you know they got it?

Teacher questioning, discussion of worked examples, the completed student tasks and the final checks in both lessons. Ask students to explain one subtask and show their correctly named saved file. Check order, termination and whether a partner can obtain the intended output. Check that students can explain their reasoning and apply the skill without relying on the worked example.

LESSON SLOTS - MONDAY & FRIDAY

Lesson	Date	Learning Objective	The Lesson Flow	Differentiation
1	24 August 2026	By the end of this lesson, students will be able to decompose a classroom computing task and follow safe working routines.	**H:** Ask how a computer-room task can go wrong without agreed routines. Invite students to explain their prediction before demonstrating the example. Use their responses to identify the idea that needs clarification. **ID:** Explain file naming, respectful use and decomposition into smaller tasks. Distinguish the overall problem from an individual task. Explain that decomposition helps a programmer organise work, locate errors and discuss responsibilities without trying to solve every detail at once. **WD:** Break preparing a class survey into input, processing and output tasks together. Ask students to identify how survey questions are collected, how answers are stored and how totals	**Support:** Provide a main-problem box with two subtasks already shown. Ask what the user does, what the computer does and what result is needed. Demonstrate saving and reopening the file before students attempt it independently. **Extension:** Students divide a subtask into further steps and identify a dependency between two parts. Ask how separate programmers could develop the parts while agreeing the information exchanged.

			are displayed. Arrange their suggestions into a diagram and discuss which tasks depend on earlier results. **YD:** Create a decomposition diagram for organising a class quiz; name each subtask and practise saving it. Students label the main problem and at least four subtasks. They exchange diagrams with a partner, explain the purpose of each part and amend a task that is too vague to implement. **C:** Ask students to explain one subtask and show their correctly named saved file. Ask students to support their answer with a specific example from their work, and record any misconception that needs a further demonstration.	
2	28 August 2026	By the end of this lesson, students will be able to write an unambiguous algorithm with ordered steps.	**H:** Give deliberately vague instructions for drawing a shape. Invite students to explain their prediction before demonstrating the example. Use their responses to identify the idea that needs clarification. **ID:** Explain an algorithm as a finite, ordered method with a clear purpose. Discuss why a computer cannot infer missing steps from everyday experience. Introduce inputs and outputs and explain that instructions must specify what happens before the process ends. **WD:** Improve instructions for calculating the total price of two items. Use prices of 15 and 20 to follow the purchase algorithm. Ask where the values enter, where the total is calculated and how the customer receives the	**Support:** Provide an instruction bank and an input-process-output template. Model the first two steps, then ask students to arrange and complete the remaining instructions. Use oral rehearsal before they write the final algorithm. **Extension:** Students identify an assumption, such as whether prices can be negative, and explain how changing that assumption would affect the algorithm. They compare two correct sequences and justify their preference.



			result. Rewrite an ambiguous instruction together. **YD:** Write an algorithm for a school purchase, then let a partner follow it literally and revise unclear steps. Students include a beginning, ordered processing steps and an identifiable output. Their partner records where an instruction could have more than one interpretation and checks the revised version using different prices. **C:** Check order, termination and whether a partner can obtain the intended output. Ask students to support their answer with a specific example from their work, and record any misconception that needs a further demonstration.	
3				Support: Extension:
4				Support: Extension:

MATERIALS NEEDED FOR THE WEEK

- Pearson Edexcel International GCSE Computer Science Student Book: Chapters 1 and 4
- Teacher computer and classroom display
- Classroom computers with Python 3 editor and individual save folders
- Prepared algorithm, tracing or Python practice tasks aligned with the lesson objectives
- Whiteboard and markers
- Student notebooks and pencils
- Success criteria, test tables and teacher observation checklist



END-OF-WEEK REFLECTION

- What worked well?

- What flopped or confused students?

- What needs to be re-taught or adjusted next week?